

Name:T.Ramyasri

H.NO:2303A52136

ASSIGNMENT-10.3

BATCH-41

PROBLEM STATEMENT 1:

Problem Statement 1: AI-Assisted Bug Detection

Scenario: A junior developer wrote the following Python function to calculate factorials:

```
def factorial(n):  
    result = 1  
    for i in range(1, n):  
        result = result * i  
    return result
```

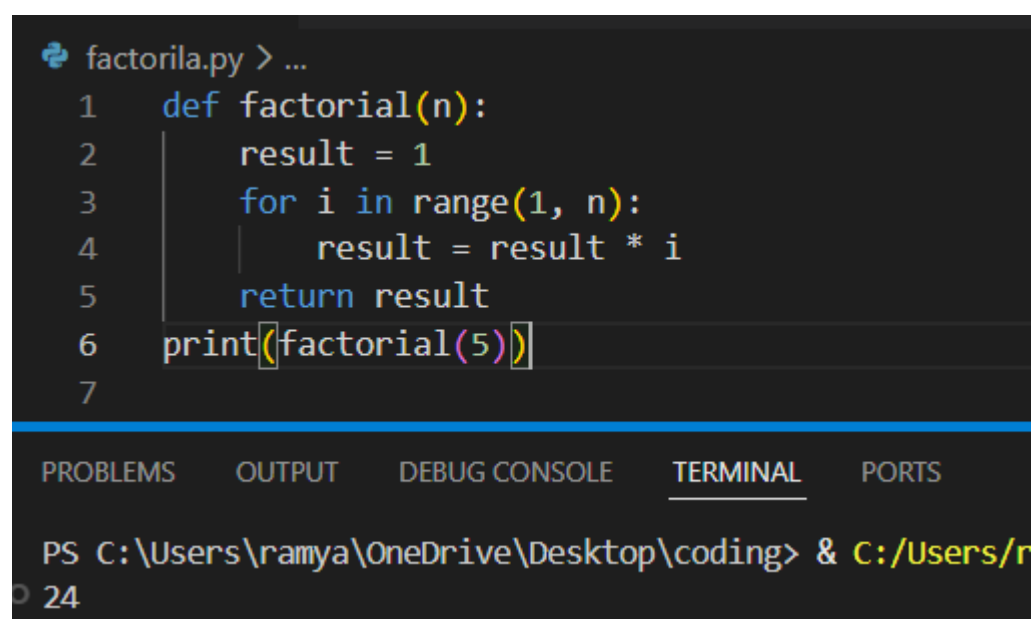
Instructions:

1. Run the code and test it with factorial(5).
2. Use an AI assistant to:
 - o Identify the logical bug in the code.
 - o Explain why the bug occurs (e.g., off-by-one error).
 - o Provide a corrected version.
3. Compare the AI's corrected code with your own manual fix.
4. Write a brief comparison: Did AI miss any edge cases (e.g., negative numbers, zero)?

Expected Output:

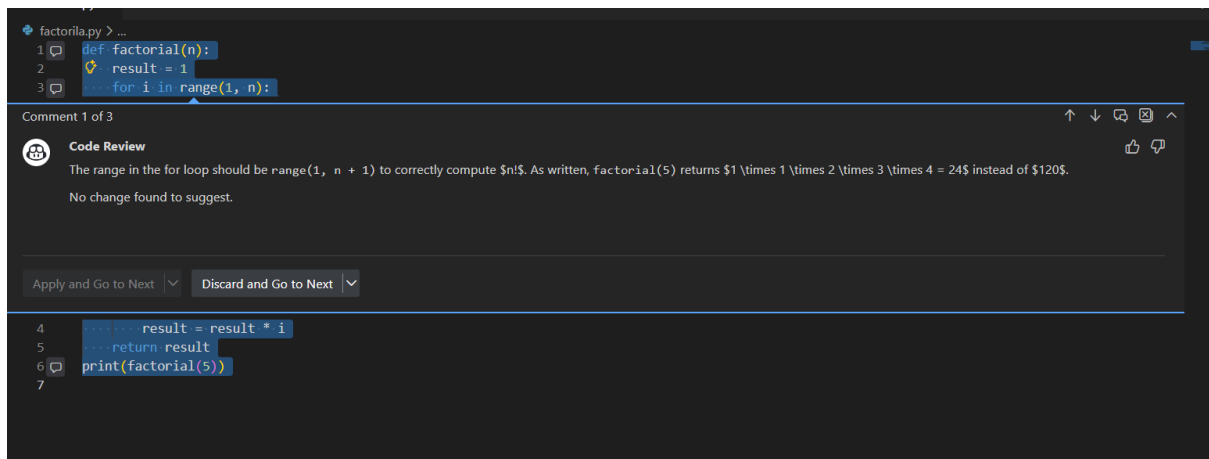
Corrected function should return 120 for factorial(5).

1.



```
factorila.py > ...  
1  def factorial(n):  
2      result = 1  
3      for i in range(1, n):  
4          result = result * i  
5      return result  
6  print(factorial(5))  
7  
  
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
  
PS C:\Users\ramya\OneDrive\Desktop\coding> & C:/Users/r  
24
```

2.



```
factorila.py > ...
1 def factorial(n):
2     result = 1
3     for i in range(1, n):
4
5         result = result * i
6     return result
7     print(factorial(5))
```

Comment 1 of 3

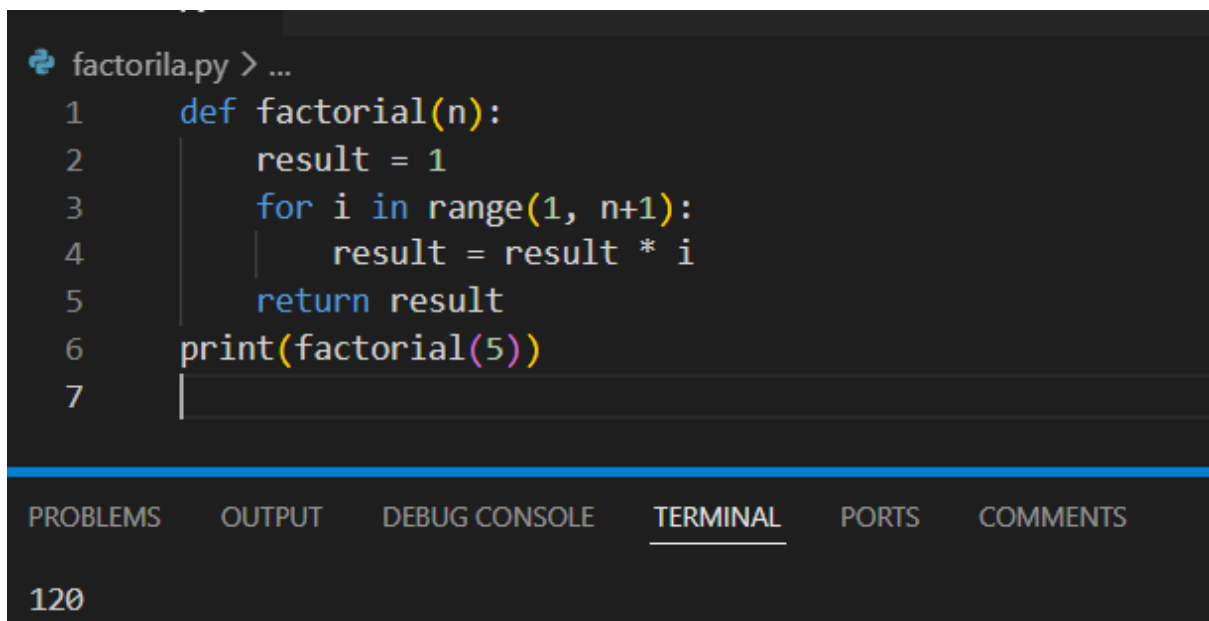
Code Review

The range in the for loop should be `range(1, n + 1)` to correctly compute $n!$. As written, `factorial(5)` returns $1 \times 2 \times 3 \times 4 = 24$ instead of 120 .

No change found to suggest.

Apply and Go to Next Discard and Go to Next

Corrected Version:



```
factorila.py > ...
1 def factorial(n):
2     result = 1
3     for i in range(1, n+1):
4         result = result * i
5     return result
6 print(factorial(5))
7
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS

120

3.Comparison between AI and Manual :

Manual Fix:

- Changed `range(1, n) → range(1, n+1)`
- Solves main logic issue.

AI Fix:

- Identified off-by-one error.
- Included `n+1` correction.
- Also handled:
 - Negative numbers
 - Zero

- Non-integer inputs

4. The AI correctly identified the **off-by-one error** caused by using `range(1, n)` instead of `range(1, n+1)`.

However, a more advanced AI-assisted correction also includes validation for:

- Negative inputs
- Non-integer inputs
- Mathematical edge case of $0! = 1$

Thus, AI-assisted debugging not only fixes logic errors but can improve robustness and reliability of the code.

JUSTIFICATION:

In this problem the code is about factorization of a number the loop runs only from starting to ending -1. if we want to know the factorial of a end number the loop should run upto end+1 times.

Problem Statement 2: Task 2 — Improving Readability & Documentation

Scenario: The following code works but is poorly written:

```
def calc(a, b, c):
    if c == "add":
        return a + b
    elif c == "sub":
        return a - b
```

```
elif c == "mul":
    return a * b
elif c == "div":
```

Instructions:

5. Use AI to:
 - Critique the function's readability, parameter naming, and lack of documentation.
 - Rewrite the function with:
 1. Descriptive function and parameter names.
 2. A complete docstring (description, parameters, return value, examples).
 3. Exception handling for division by zero.
 4. Consideration of input validation.
6. Compare the original and AI-improved versions.
7. Test both with valid and invalid inputs (e.g., division by zero, non-string operation).

Expected Output:

A well-documented, robust, and readable function that handles errors gracefully.

5.

```
def calc(a, b, c):  
    if c == "add":  
        return a + b  
    elif c == "sub":  
        return a - b  
    elif c == "mul":  
        return a * b  
    elif c == "div":  
        return a / b
```

1. AI Critique of the Original Code

1. Poor Readability

- Function name calc is vague.
- Parameter names a, b, c are not descriptive.
- No comments or documentation.
- No explanation of expected inputs.

2. No Documentation

- No docstring explaining:
 - What the function does
 - What parameters mean
 - What it returns
 - Example usage

3. No Error Handling

- Division by zero will crash the program.
- If operation is invalid (e.g., "power"), function returns None.
- No type checking for inputs.

4. No Input Validation

- What if numbers are strings?
- What if operation is not a string?

2. AI Improved version:

```
factorila.py > ...
1 def calculate(num1, num2, operation):
2     """
3     Perform a basic arithmetic operation on two numbers.
4
5     Parameters:
6     -----
7     num1 : int or float
8         The first number.
9     num2 : int or float
10        The second number.
11    operation : str
12        The arithmetic operation to perform.
13        Supported values:
14        - "add" : Addition
15        - "sub" : Subtraction
16        - "mul" : Multiplication
17        - "div" : Division
18
19    Returns:
20    -----
21    float or int
22        The result of the arithmetic operation.
23
24    Raises:
25    -----
26    TypeError:
27        If inputs are of incorrect type.
28    ValueError:
29        If operation is unsupported.
30    ZeroDivisionError:
31        If division by zero is attempted.
32
33    Examples:
34    -----
35    >>> calculate(5, 3, "add")
36    8
37
38    >>> calculate(10, 2, "div")
39    5.0
40    """
41
42    # Input validation
43    if not isinstance(num1, (int, float)):
44        raise TypeError("num1 must be an integer or float.")
```

```

if not isinstance(num1, (int, float)):
    raise TypeError("num1 must be an integer or float.")

if not isinstance(num2, (int, float)):
    raise TypeError("num2 must be an integer or float.")

if not isinstance(operation, str):
    raise TypeError("operation must be a string.")

operation = operation.lower()

if operation == "add":
    return num1 + num2

elif operation == "sub":
    return num1 - num2

elif operation == "mul":
    return num1 * num2

elif operation == "div":
    if num2 == 0:
        raise ZeroDivisionError("Cannot divide by zero.")
    return num1 / num2

else:
    raise ValueError("Unsupported operation. Use add, sub, mul, or div.")

```

Comparison:

Comparison – Original vs Improved Version

Feature	Original Version	AI-Improved Version
Function Name	✗ calc (unclear)	✓ calculate (descriptive)
Parameter Names	✗ a, b, c	✓ num1, num2, operation
Documentation	✗ None	✓ Complete docstring
Error Handling	✗ None	✓ Division by zero handled
Invalid Operation	✗ Returns None	✓ Raises ValueError
Input Validation	✗ No checks	✓ Type checking added
Readability	✗ Low	✓ High

IMPROVED:

```
69         raise ValueError("Unsupported operation. Use add, sub, mul, or div.")
70
71     print(calculate(10, 5, "add"))    # Improved
72
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS

```
PS C:\Users\ramya\OneDrive\Desktop\coding> & C:/Users/ramya/AppData/Local/Microsoft/Windows
PS C:\Users\ramya\OneDrive\Desktop\coding> ^C
PS C:\Users\ramya\OneDrive\Desktop\coding> & C:/Users/ramya/AppData/Local/Microsoft/Windows
15
PS C:\Users\ramya\OneDrive\Desktop\coding> ^C
PS C:\Users\ramya\OneDrive\Desktop\coding> & C:/Users/ramya/AppData/Local/Microsoft/Windows
15
PS C:\Users\ramya\OneDrive\Desktop\coding>
```

```
68     else:
69         raise ValueError("Unsupported operation. Use add, sub, mul, or div.")
70     print(calculate(10, 0, "div"))
71
72
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS

```
PS C:\Users\ramya\OneDrive\Desktop\coding> & C:/Users/ramya/AppData/Local/Microsoft/Windows
15 ...
PS C:\Users\ramya\OneDrive\Desktop\coding> & C:/Users/ramya/AppData/Local/Microsoft/Windows
15
PS C:\Users\ramya\OneDrive\Desktop\coding> & C:/Users/ramya/AppData/Local/Microsoft/Windows
Traceback (most recent call last):
  File "c:\Users\ramya\OneDrive\Desktop\coding\factorila.py", line 70, in <module>
    print(calculate(10, 0, "div"))
    ^
  File "c:\Users\ramya\OneDrive\Desktop\coding\factorila.py", line 65, in calculate
    raise ZeroDivisionError("Cannot divide by zero.")
ZeroDivisionError: Cannot divide by zero.
PS C:\Users\ramya\OneDrive\Desktop\coding>
```

```
66         return num1 / num2
67
68     else:
69         raise ValueError("Unsupported operation. Use add, sub, mul, or div.")
70 print(calculate(10, 5, "power"))
71
72
73
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS

PS C:\Users\ramya\OneDrive\Desktop\coding> & C:/Users/ramya/AppData/Local/Microsoft/WindowsApps/p
raise ZeroDivisionError("Cannot divide by zero.")
ZeroDivisionError: Cannot divide by zero.

PS C:\Users\ramya\OneDrive\Desktop\coding> & C:/Users/ramya/AppData/Local/Microsoft/WindowsApps/p
Traceback (most recent call last):
File "c:\Users\ramya\OneDrive\Desktop\coding\factorila.py", line 70, in <module>
print(calculate(10, 5, "power"))
^^
File "c:\Users\ramya\OneDrive\Desktop\coding\factorila.py", line 69, in calculate
raise ValueError("Unsupported operation. Use add, sub, mul, or div.")
ValueError: Unsupported operation. Use add, sub, mul, or div.

```
69         raise ValueError("Unsupported operation. Use add, sub, mul, or div.")
70 print(calculate(10, 5, 123)) print(calculate(10, 5, "div"))
71
72
73
74
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS

PS C:\Users\ramya\OneDrive\Desktop\coding> & C:/Users/ramya/AppData/Local/Microsoft/WindowsApps/p
raise ValueError("Unsupported operation. Use add, sub, mul, or div.")
ValueError: Unsupported operation. Use add, sub, mul, or div.

PS C:\Users\ramya\OneDrive\Desktop\coding> & C:/Users/ramya/AppData/Local/Microsoft/WindowsApps/p
Traceback (most recent call last):
File "c:\Users\ramya\OneDrive\Desktop\coding\factorila.py", line 70, in <module>
print(calculate(10, 5, 123))
^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "c:\Users\ramya\OneDrive\Desktop\coding\factorila.py", line 50, in calculate
raise TypeError("operation must be a string.")
TypeError: operation must be a string.

PS C:\Users\ramya\OneDrive\Desktop\coding>

ORIGINAL CODE:


```
factorila.py > ...
1 def calc(a, b, c):
2     if c == "add":
3         return a + b
4     elif c == "sub":
5         return a - b
6     elif c == "mul":
7         return a * b
8     elif c == "div":
9         return a / b
10 print(calc(10, 5, "add"))
11 print(calc(10, 0, "div"))
12 print(calc(10, 5, "power"))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS
PS C:\Users\ramya\OneDrive\Desktop\coding> & C:/Users/ramya/AppData/Local/Microsoft/WindowsApps/python3.11/python.exe factorila.py
PS C:\Users\ramya\OneDrive\Desktop\coding> & C:/Users/ramya/AppData/Local/Microsoft/WindowsApps/python3.11/python.exe factorila.py
15
Traceback (most recent call last):
  File "c:\Users\ramya\OneDrive\Desktop\coding\factorila.py", line 11, in <module>
    print(calc(10, 0, "div"))
    ~~~~~^~~~~
  File "c:\Users\ramya\OneDrive\Desktop\coding\factorila.py", line 9, in calc
    return a / b
    ~~~~~^~~~~
ZeroDivisionError: division by zero
PS C:\Users\ramya\OneDrive\Desktop\coding>
```

JUSTIFICATION:

The original version works for ideal inputs but fails under invalid conditions. The AI-enhanced version handles errors gracefully and follows professional coding standards.

Problem Statement 3: Enforcing Coding Standards

Scenario: A team project requires PEP8 compliance. A developer submits:

```
def Checkprime(n):  
    for i in range(2, n):  
        if n % i == 0:  
            return False  
    return True
```

Instructions:

8. Verify the function works correctly for sample inputs.
9. Use an AI tool (e.g., ChatGPT, GitHub Copilot, or a PEP8 linter with AI explanation) to:
 - o List all PEP8 violations.
 - o Refactor the code (function name, spacing, indentation, naming).
10. Apply the AI-suggested changes and verify functionality is preserved.

11. Write a short note on how automated AI reviews could streamline code reviews in large teams.

Expected Output:

A PEP8-compliant version of the function, e.g.:

```
def check_prime(n):  
    for i in range(2, n):  
        if n % i == 0:  
            return False  
    return True
```

```
1  def Checkprime(n):  
2      for i in range(2, n):  
3          if n % i == 0:  
4              return False  
5      return True  
6
```

Test case:

```
1 def Checkprime(n):
2     for i in range(2, n):
3         if n % i == 0:
4             return False
5     return True
6 print(Checkprime(7))    # Prime
7 print(Checkprime(10))  # Not prime
8 print(Checkprime(2))   # Prime
9
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS

```
PS C:\Users\ramya\OneDrive\Desktop\coding> & C:/Users/ramya/AppData/Local/Microsoft/Windows/In...
```

```
PS C:\Users\ramya\OneDrive\Desktop\coding> & C:/Users/ramya/AppData/Local/Micro
True
False
True
```

```
factorila.py > ...
1 def Checkprime(n):
2     for i in range(2, n):
3         if n % i == 0:
4             return False
5     return True
6 print(Checkprime(7)) # Prime
7 print(Checkprime(10)) # Not prime
8 print(Checkprime(2)) # Prime
9 print(Checkprime(1))
10
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS COMMENTS

```
PS C:\Users\ramya\OneDrive\Desktop\coding> & C:/Users/ramya/AppData/Local/Microsoft/WindowsApps/python3.11.exe
PS C:\Users\ramya\OneDrive\Desktop\coding> & C:/Users/ramya/AppData/Local/Microsoft/WindowsApps/python3.11.exe
True
False
True
PS C:\Users\ramya\OneDrive\Desktop\coding> & C:/Users/ramya/AppData/Local/Microsoft/WindowsApps/python3.11.exe
True
False
True
PS C:\Users\ramya\OneDrive\Desktop\coding> & C:/Users/ramya/AppData/Local/Microsoft/WindowsApps/python3.11.exe
True
False
True
PS C:\Users\ramya\OneDrive\Desktop\coding> & C:/Users/ramya/AppData/Local/Microsoft/WindowsApps/python3.11.exe
True
False
True
True
```

PEP8 Violations Identified

According to **PEP8 (Python Style Guide)**:

Violations in Original Code

- ❌ Function name Checkprime
 - Should use **snake_case** → check_prime
- ❌ Missing docstring
 - Functions should include documentation.
- ❌ No input validation
 - Should check if n is integer and > 1.
- ❌ Inefficient loop

- Checking up to $\sqrt{n}$ instead of $\sqrt[3]{n}$.
- 5. ❌ No spacing after function definition line (readability issue in formatting submissions)
- 6. ❌ No blank line before function (PEP8 requires spacing between top-level functions)

AI-Refactored PEP8-Compliant Version

```
factorila.py > ...
1  def check_prime(n):
2      """
3      Determine whether a number is prime.
4
5      Parameters:
6      -----
7      n : int
8      |     The number to check.
9
10     Returns:
11     -----
12     bool
13     |     True if n is a prime number, False otherwise.
14
15     Raises:
16     -----
17     TypeError:
18     |     If n is not an integer.
19     """
20
21     if not isinstance(n, int):
22         raise TypeError("Input must be an integer.")
23
24     if n <= 1:
25         return False
26
27     for i in range(2, int(n ** 0.5) + 1):
28         if n % i == 0:
29             return False
30
31     return True
32
33 print(check_prime(7))    # True
34 print(check_prime(10))  # False
35 print(check_prime(2))   # True
36 print(check_prime(1))   # False
37 print(check_prime(0))   # False
38 print(check_prime(-5))  # False
39 print(check_prime(3.5)) # TypeError
```

```
PS C:\Users\ramya\OneDrive\Desktop\coding> & C:/Users/ramya/AppData/Local/Microsoft/W
● PS C:\Users\ramya\OneDrive\Desktop\coding> & C:/Users/ramya/AppData/Local/Microsoft/W
True
False
True
False
False
False
False
○ PS C:\Users\ramya\OneDrive\Desktop\coding>
```

Comparison – Original vs Refactored

Feature	Original	Refactored
Function Name	✗ Checkprime	✓ check_prime
PEP8 Compliant	✗ No	✓ Yes
Docstring	✗ None	✓ Added
Edge Case Handling	✗ No	✓ Yes
Input Validation	✗ No	✓ Type check added
Efficiency	✗ O(n)	✓ O(√n)
Readability	✗ Basic	✓ Professional

Role of AI in Code Reviews

Automated AI-based code reviews can significantly streamline development in large teams by:

- 1. Instant PEP8 Compliance Checks**
AI tools automatically detect naming issues, spacing problems, and formatting violations.
- 2. Early Bug Detection**
Logical errors and missing edge cases can be identified before manual review.
- 3. Improved Code Quality**
AI suggests optimizations (e.g., checking up to $\sqrt{n}$ instead of n).
- 4. Reduced Review Time**
Senior developers can focus on architecture instead of style corrections.
- 5. Consistent Coding Standards**
AI ensures uniform coding style across the team.

6. Educational Benefit

Junior developers learn best practices through AI-generated explanations.

```
def check_prime(n):
    if not isinstance(n, int):
        raise TypeError("Input must be an integer.")

    if n <= 1:
        return False

    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False

def factorial(n):
```

Justification:

The original function worked for basic inputs but violated **PEP8 coding standards**, lacked proper naming conventions, documentation, and edge-case handling.

The refactored version improves:

- ✓ **Readability** (snake_case naming: check_prime)
- ✓ **Maintainability** (clear structure and formatting)
- ✓ **Robustness** (handles invalid inputs and edge cases like $n \leq 1$)
- ✓ **Efficiency** (checks up to $\sqrt{n}$ instead of n)

Using AI-assisted review helps enforce coding standards automatically, reduces manual review effort, and improves overall code quality in large team projects.

Problem Statement 4: AI as a Code Reviewer in Real Projects

Scenario:

In a GitHub project, a teammate submits:

```
def processData(d):  
    return [x * 2 for x in d if x % 2 == 0]
```

Instructions:

1. Manually review the function for:
 - Readability and naming.
 - Reusability and modularity.
 - Edge cases (non-list input, empty list, non-integer elements).
2. Use AI to generate a code review covering:
 - a. Better naming and function purpose clarity.
 - b. Input validation and type hints.
 - c. Suggestions for generalization (e.g., configurable multiplier).
3. Refactor the function based on AI feedback.
4. Write a short reflection on whether AI should be a standalone reviewer or an assistant.

Expected Output:

An improved function with type hints, validation, and clearer intent, e.g.:

```
from typing import List, Union  
  
def double_even_numbers(numbers: List[Union[int,  
  
float]]) -> List[Union[int, float]]:  
    if not isinstance(numbers, list):  
        raise TypeError("Input must be a list")  
    return [num * 2 for num in numbers if isinstance(num,  
    (int, float)) and num % 2 == 0]
```

```
factorila.py > ...  
1  def processData(d):  
2      return [x * 2 for x in d if x % 2 == 0]  
3
```

1. Readability & Naming Issues

- ❌ Function name `processData` is vague.

- ❌ Parameter name `d` is unclear.
- ❌ No docstring explaining purpose.
- ❌ No type hints.

Better naming should reflect intent, such as:
`double_even_numbers(numbers)`

2. Reusability & Modularity

- Multiplier 2 is hardcoded.
- Not reusable for other multipliers (e.g., triple even numbers).
- Not modular — lacks flexibility.
- **3. Edge Cases**

Case	Problem
Non-list input	Will raise unexpected error
Empty list	Works (returns empty list) ✅
Non-integer elements	May raise error (<code>x % 2</code>)
String elements	Crashes
None input	Crashes

AI-Generated Code Review Suggestions

◆ Improvements Suggested by AI

a) Better Naming

- Rename function to reflect behavior clearly.
- Use descriptive parameter names.

b) Add Type Hints

- Improve clarity and static analysis support.

c) Input Validation

- Ensure input is a list.
- Ensure elements are numeric.

d) Generalization

- Make multiplier configurable.
- Improve flexibility.

from typing import List, Union

Refactored Version (AI-Improved)

```

factorila.py > ...
1  from typing import List, Union
2
3
4  def multiply_even_numbers(
5      numbers: List[Union[int, float]],
6      multiplier: Union[int, float] = 2
7  ) -> List[Union[int, float]]:
8      """
9      Multiply all even numbers in a list by a given multiplier.
10
11      Parameters:
12      -----
13      numbers : List[int or float]
14      | A list of numeric values.
15      multiplier : int or float, optional
16      | The value by which even numbers will be multiplied.
17      | Default is 2.
18
19      Returns:
20      -----
21      List[int or float]
22      | A new list containing the modified even numbers.
23
24      Raises:
25      -----
26      TypeError:
27      | If numbers is not a list or contains non-numeric elements.
28      """
29
30      if not isinstance(numbers, list):
31          raise TypeError("Input must be a list.")
32
33      result = []
34
35      for num in numbers:
36          if not isinstance(num, (int, float)):
37              raise TypeError("All elements in the list must be numeric.")
38
39          if num % 2 == 0:
40              result.append(num * multiplier)
41
42      return result
43  print(multiply_even_numbers([1, 2, 3, 4]))
44  print(multiply_even_numbers([2, 4, 6], multiplier=3))
45  multiply_even_numbers("1234")
46  multiply_even_numbers([2, "a", 4])
47  multiply_even_numbers([2, "a", 4])
48

```

```

PS C:\Users\ramya\OneDrive\Desktop\coding> & C:/Users/ramya/AppData/Local/Microsoft/WindowsApps/python
[4, 8]
[6, 12, 18]
Traceback (most recent call last):
  File "c:\Users\ramya\OneDrive\Desktop\coding\factorila.py", line 45, in <module>
    multiply_even_numbers("1234")
  File "c:\Users\ramya\OneDrive\Desktop\coding\factorila.py", line 31, in multiply_even_numbers
    raise TypeError("Input must be a list.")
TypeError: Input must be a list.
PS C:\Users\ramya\OneDrive\Desktop\coding>

```

Comparison – Original vs Improved

Feature	Original	Improved
Naming	✗ Vague	✓ Clear & descriptive
Type Hints	✗ None	✓ Added
Input Validation	✗ No	✓ Yes
Flexibility	✗ Fixed multiplier	✓ Configurable
Documentation	✗ None	✓ Full docstring
Error Handling	✗ None	✓ Explicit

JUSTIFICATION:

AI works best as a **first-line automated reviewer**, while humans handle architectural decisions and deeper logic validation.

Problem Statement 5: — AI-Assisted Performance Optimization

Scenario: You are given a function that processes a list of integers, but it runs slowly on large datasets:

```
def sum_of_squares(numbers):
    total = 0
    for num in numbers:
        total += num ** 2
    return total
```

Instructions:

- Test the function with a large list (e.g., `range(1000000)`).
- Use AI to:
 - Analyze time complexity.
 - Suggest performance improvements (e.g., using built-in functions, vectorization with NumPy if applicable).
 - Provide an optimized version.
- Compare execution time before and after optimization.
- Discuss trade-offs between readability and performance.

Expected Output:

An optimized function, such as:

```
def sum_of_squares_optimized(numbers):
    return sum(x * x for x in numbers)
```

ORIGINAL FUNCTION:

```
def sum_of_squares(numbers):
    total = 0
    for num in numbers:
        total += num ** 2
    return total
```

```
factorila.py > ...
1 def sum_of_squares(numbers):
2     total = 0
3     for num in numbers:
4         total += num ** 2
5     return total
6 numbers = range(1_000_000)
7 print(sum_of_squares(numbers))
8
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS

```
PS C:\Users\ramya\OneDrive\Desktop\coding> & C:/Users/ramya/AppData/
torila.py
333332833333500000
```

Time Complexity Analysis

- The function iterates through all elements once.
- Each operation ($\text{num} ** 2$) is $O(1)$.
- Therefore:

✓ Time Complexity: $O(n)$

✓ Space Complexity: $O(1)$ (constant extra space)

The complexity cannot be reduced below $O(n)$ because every element must be processed.

However, we can optimize **implementation efficiency**.

AI-Suggested Performance Improvements

- ◆ **Improvement 1: Use Built-in `sum()` with Generator Expression**

Python's built-in functions are implemented in C and are faster.

```
1 def sum_of_squares_optimized(numbers):
2     return sum(x * x for x in numbers)
3
```

List Comprehension (slightly less memory efficient)

```
def sum_of_squares_list(numbers):  
    return sum([x * x for x in numbers])
```

Improvement 3 (Advanced): Using NumPy (for very large datasets)

```
import numpy as np  
  
def sum_of_squares_numpy(numbers):  
    arr = np.array(numbers)  
    return np.sum(arr ** 2)  
  
def sum_of_squares_loop(numbers):
```

Trade-offs Between Readability and Performance

Approach	Readability	Performance	Memory Usage
Original Loop	Medium	Good	Low
Generator + sum()	High	Better	Low
List Comprehension	High	Good	Higher
NumPy	Medium	Best (large data)	Higher

JUSTIFICATION:

- ☐ The original function has optimal time complexity $O(n)$.
- ☐ Performance improvement comes from using optimized built-in functions.
- ☐ AI-assisted optimization helps:
 - Improve readability
 - Suggest Pythonic solutions
 - Recommend advanced tools like NumPy
 - Balance performance vs maintainability